Bases de données NoSQL

BUT Science des Données - Metz

Berhocoïrigoin Elena

Avant-propos

- Migration Relationnel - NoSQL
  - Un seul TD 2h
  - Challenge étudiants BUT de Dole en janvier
- N'hésitez pas à poser des questions avant, pendant, après

Objectifs

- Bases de données NoSQL : qu'est-ce que c'est ?
- À quels besoins répondent-elles ?
- Introduction à deux technologies NoSQL :
  - MongoDB
  - Neo4j

Évaluation

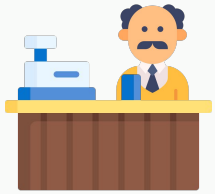
- Un examen de 2 heures (dernière séance)
- Première partie : Questions générales, papier
- Seconde partie : MongoDB / Neo4j, machine

Chapitre 1. Du modèle relationnel au NoSQL

Chapitre 2. NoSQL : les concepts

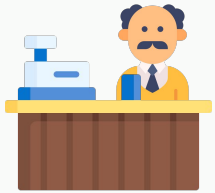
Chapitre 3. NoSQL : les catégories

Histoire d’un libraire



« Pour gérer ma librairie, j’ai besoin de suivre les stocks. »

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

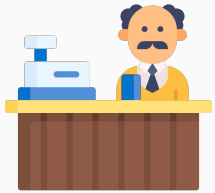
Date	ID	Quantité	Total

M. Dupont est satisfait, le tableur est :

- **Accessible** : options gratuites
- **Intuitif** : interface graphique, tout à la souris
- **Pratique** : toutes les informations sont au même endroit

« Avec Excel, tout est stocké dans un même fichier : quelle simplicité ! »

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

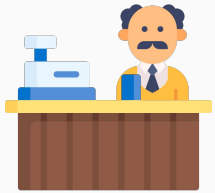
Date	ID	Quantité	Total

Oui mais voilà, la librairie a du succès ! Le fichier grossit de plus en plus et M. Dupont embauche une employée pour l'aider.

La gestion des données se complexifie à cause de :

- La **quantité de données** : lenteur, manque de lisibilité, recherches fastidieuses

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

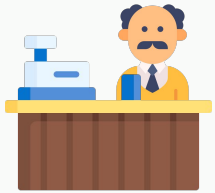
Date	ID	Quantité	Total

Oui mais voilà, la librairie a du succès ! Le fichier grossit de plus en plus et M. Dupont embauche une employée pour l'aider.

La gestion des données se complexifie à cause de :

- La quantité de données
- L'**absence de contrôle à la saisie** des données : les **erreurs silencieuses** se multiplient, d'autant plus avec la **co-édition**

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

Date	ID	Quantité	Total

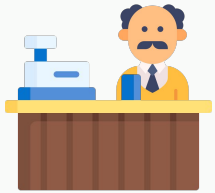
Oui mais voilà, la librairie a du succès ! Le fichier grossit de plus en plus et M. Dupont embauche une employée pour l'aider.

La gestion des données se complexifie à cause de :

- La quantité de données
- L'**absence de contrôle à la saisie** des données : les **erreurs silencieuses** se multiplient, d'autant plus avec la **co-édition**

  Exemple : Une maison d'édition vient d'être rachetée, M. Dupont modifie son nom dans le tableur mais oublie certains livres, des incohérences apparaissent. M. Dupont se trompe de colonne et met un chiffre dans une colonne de texte, aucune erreur n'est levée.

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

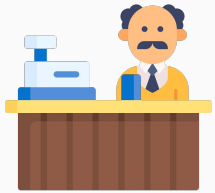
Date	ID	Quantité	Total

M. Dupont souhaiterait connaître les tendances d'achat afin de mieux anticiper les ruptures de stocks et comprendre les goûts de sa clientèle.

De nouvelles limitations apparaissent :

- Effectuer des **requêtes complexes** est **laborieux**

Une première solution : le tableur



ID	Titre	Auteur	Éditeur	…	Prix	Stock

Date	ID	Quantité	Total

Avantages :

- Accessible
- Intuitif
- Pratique

Inconvénients :

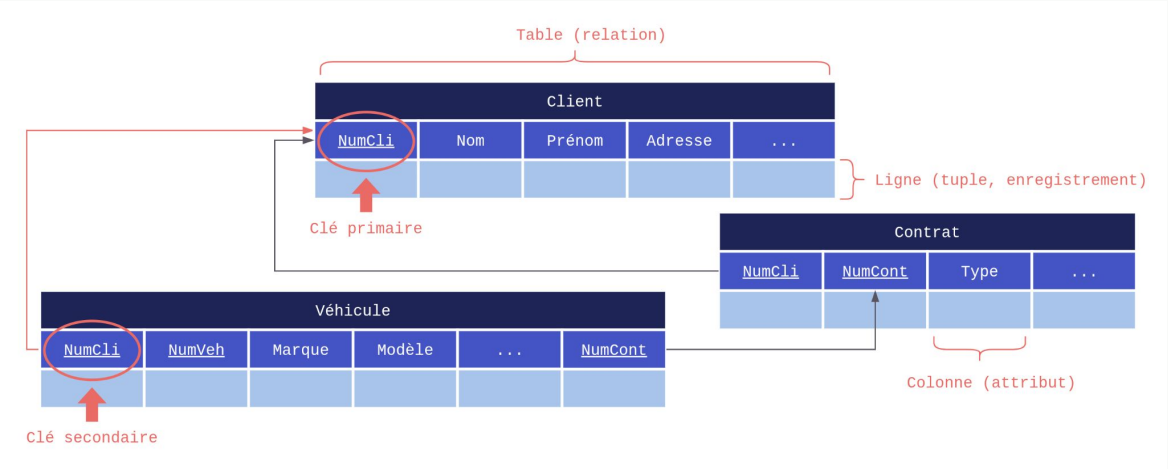
- Non adapté aux gros volumes de données
- Aucun contrôle à la saisie
- Capacité de filtrage et de recherche limitée

« C'était pratique au départ … mais c'est devenu ingérable. »

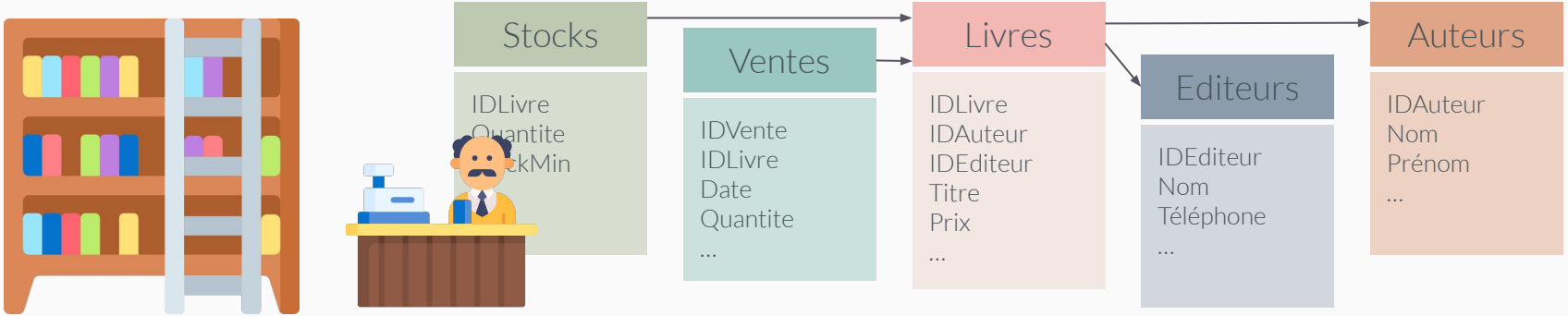
Du relationnel au NoSQL

1970 - 2000 : Domination du modèle relationnel

Choix par défaut pour le stockage de données :



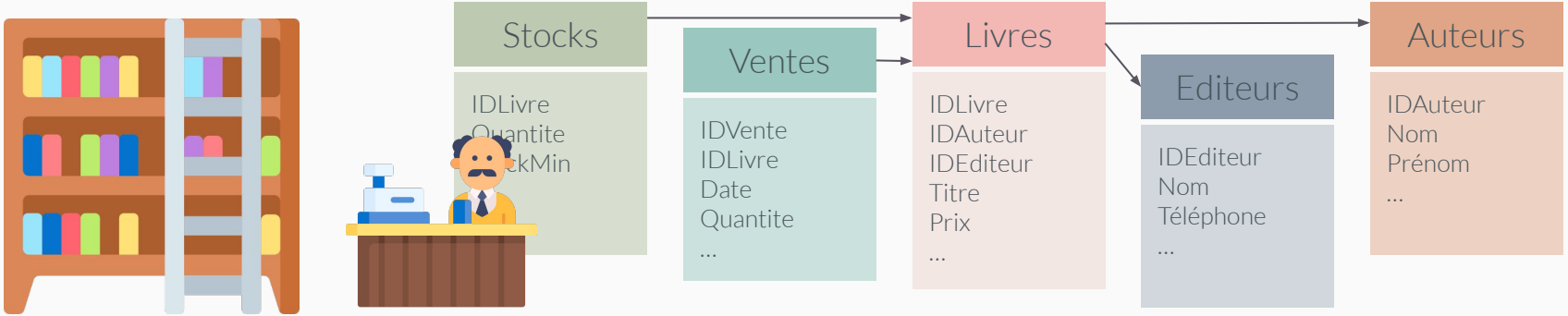
Du tableur à la base de données relationnelle



M. Dupont est ravi, la base de données relationnelle offre :

- Un **stockage structuré** : les données sont organisées en tables, reliées entre elles par des clés

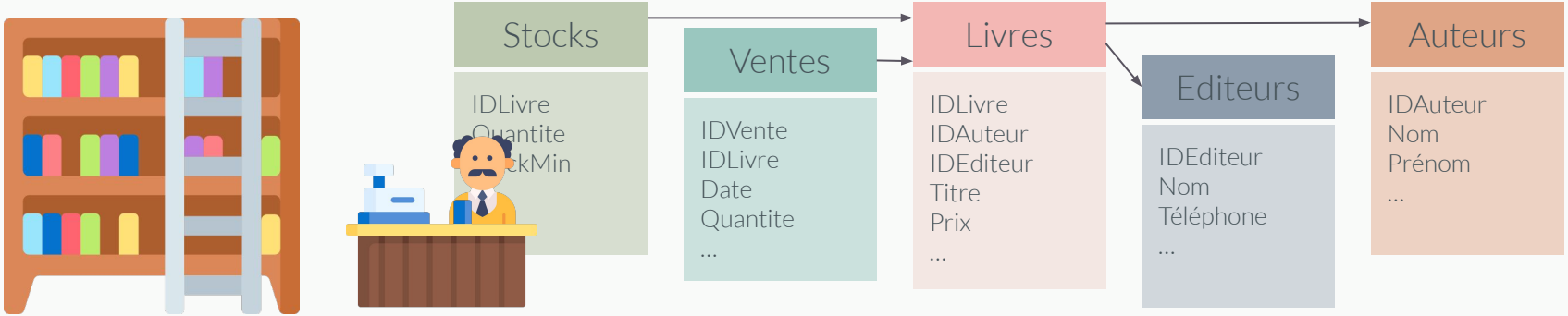
Du tableur à la base de données relationnelle



M. Dupont est ravi, la base de données relationnelle offre :

- Un **stockage structuré**
- Un **stockage efficace** : chaque information est stockée une seule fois

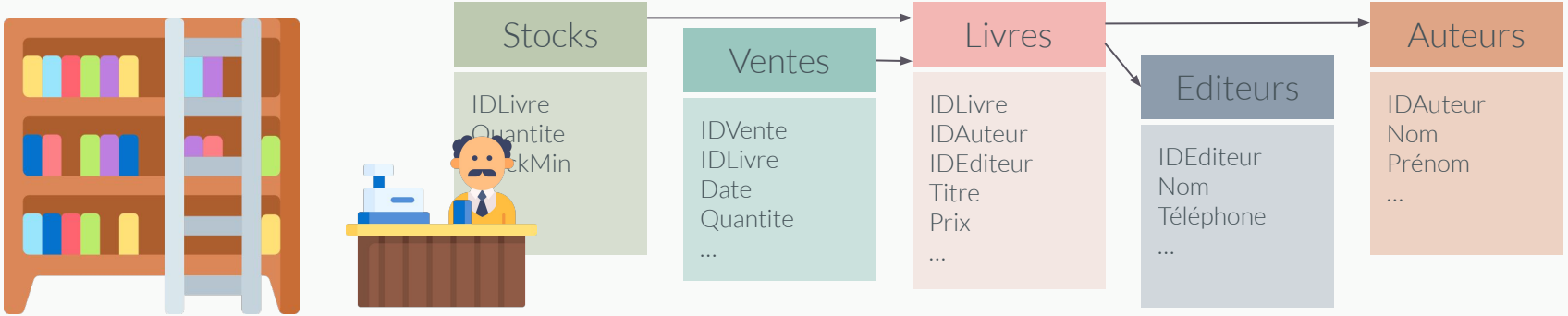
Du tableur à la base de données relationnelle



M. Dupont est ravi, la base de données relationnelle offre :

- Un **stockage structuré**
- Un **stockage efficace**
- Un **stockage fiable** : l'intégrité des données est garantie par l'absence de doublons, le contrôle à la saisie et la gestion des accès simultanés à la base

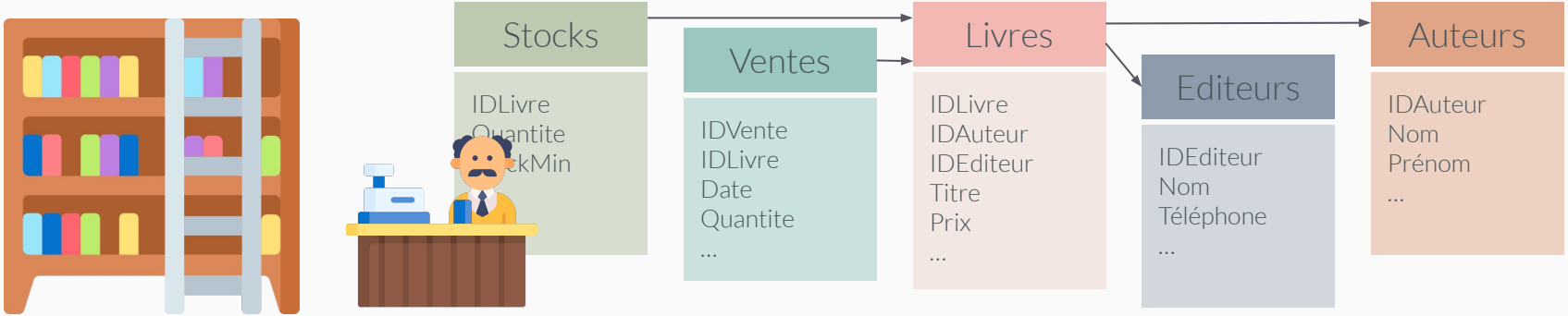
Du tableur à la base de données relationnelle



M. Dupont est ravi, la base de données relationnelle offre :

- Un **stockage structuré**
- Un **stockage efficace**
- Un **stockage fiable**
- Un **langage de requête** standardisé : SQL permet d'effectuer des requêtes sophistiquées avec seulement quelques instructions

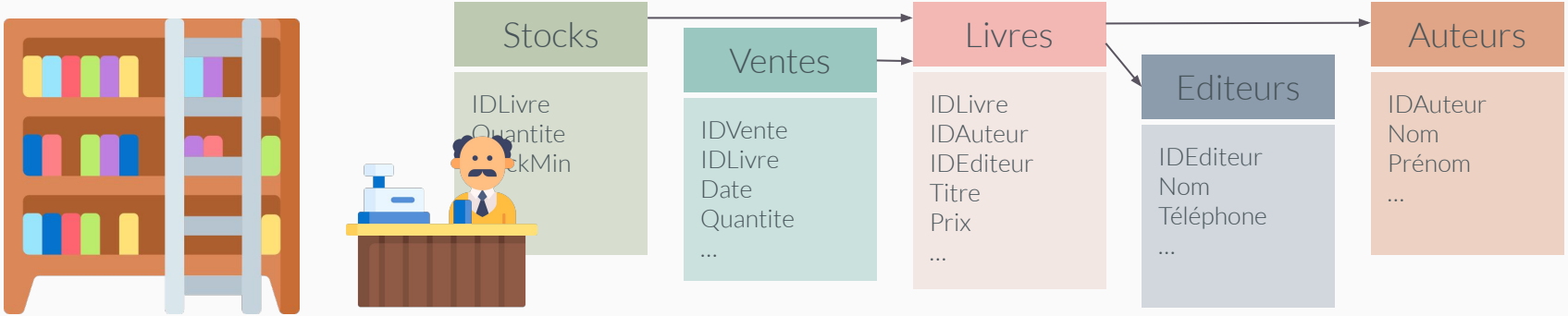
Du tableur à la base de données relationnelle



M. Dupont est ravi, la base de données relationnelle offre :

- Un **stockage structuré**
- Un **stockage efficace**
- Un **stockage fiable**
- Un **langage de requête** standardisé
- Et est capable de s'adapter à l'augmentation du nombre de livres et de ventes

Du tableur à la base de données relationnelle



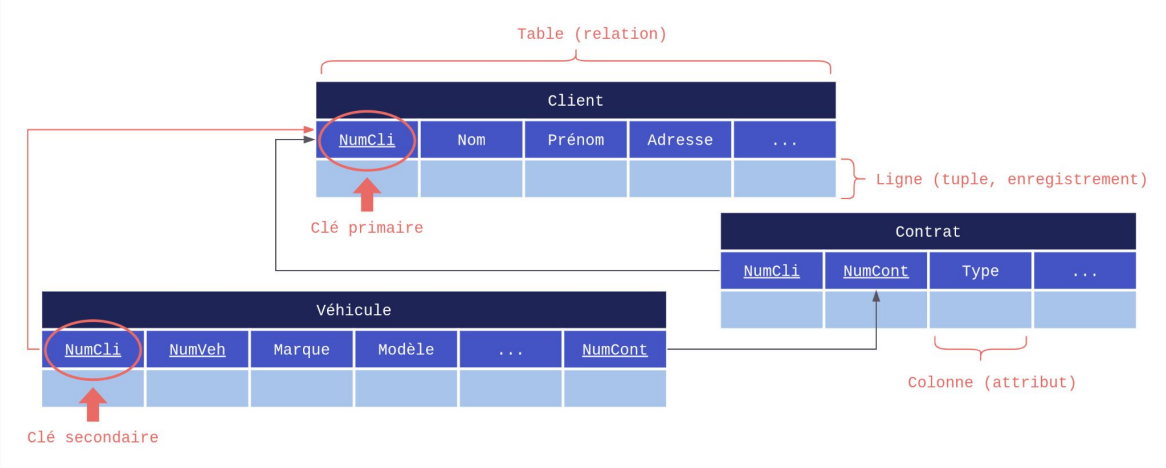
« Désormais, les données sont bien organisées et facilement interrogeables.
Je n'ai plus d'inquiétude quant à leur fiabilité non plus ! »

Du relationnel au NoSQL

1970 - 2000 : Domination du modèle relationnel

Choix par défaut pour le stockage de données :

- **Stockage structuré** (tables, clés), **efficace** (normalisation) et **fiable** (ACID)
- Requêtes facilitées par le **langage SQL** (Structured Query Language)
- Nombreux outils, documentation riche, vaste communauté d'utilisateurs



Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

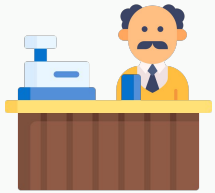
- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Volume : Données massive

Variété : Grande variété de données (ex : image, texte libre, son, vidéo, carte géographique, simulation générée par ordinateur …)

Vélocité : Vitesse à laquelle les données sont créées et doivent être stockées (stockage) et analysées (traitement) (ex : temps réel)

Du tableur à la base de données relationnelle

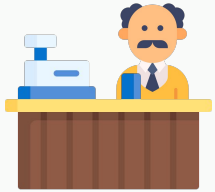


Seulement voilà, l'affaire marche tellement bien que M. Dupont décide d'étendre son **activité en ligne** … Le succès du site web dépasse les attentes ! Il enregistre un **trafic** important et génère beaucoup plus de **données** que le magasin physique.

Le **serveur** qui héberge la base de données commence à montrer des signes de faiblesses :

- Aux heures de pointe, le site est **lent** à charger les pages produits et à finaliser les commandes
- L'**espace de stockage** est **saturé**

Du tableur à la base de données relationnelle



Le libraire fait alors évoluer son infrastructure en investissant dans une machine plus puissante … mais la croissance du e-commerce est si rapide que M. Dupont se retrouve dans la même situation quelques mois plus tard. Or, acheter des **serveurs toujours plus puissants** a plusieurs inconvénients :

- Coût
- Nécessité d'**interrompre le service** en ligne
- Plafond technique

Du relationnel au NoSQL

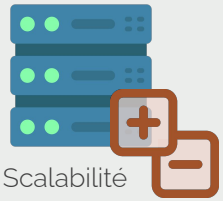
À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

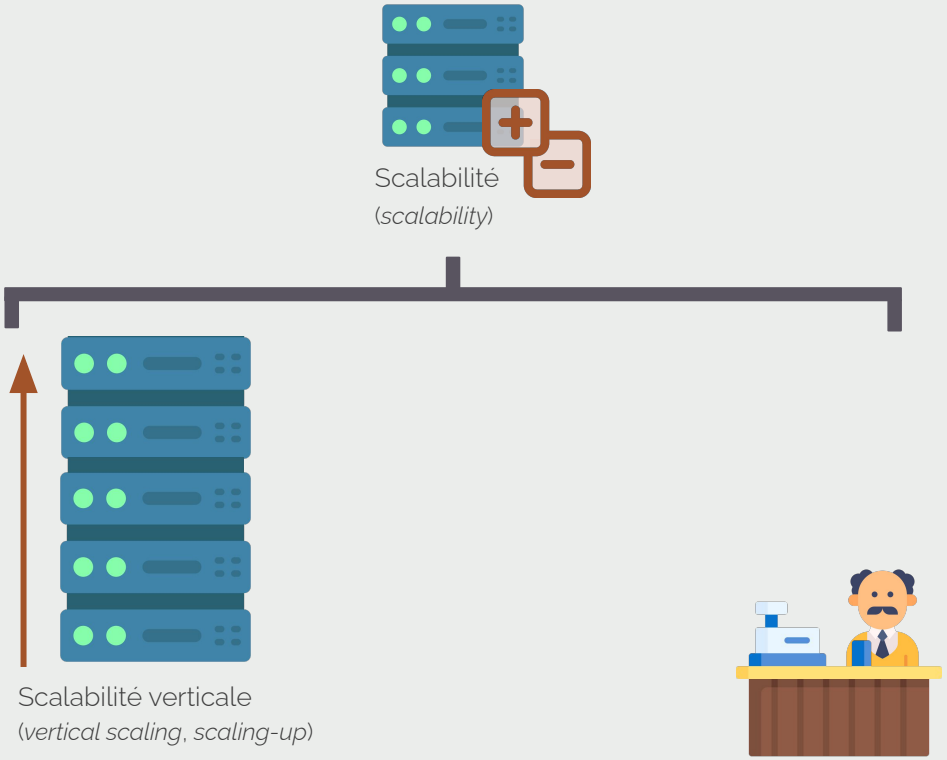
- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant



Scalabilité
(*scalability*)

Scalabilité

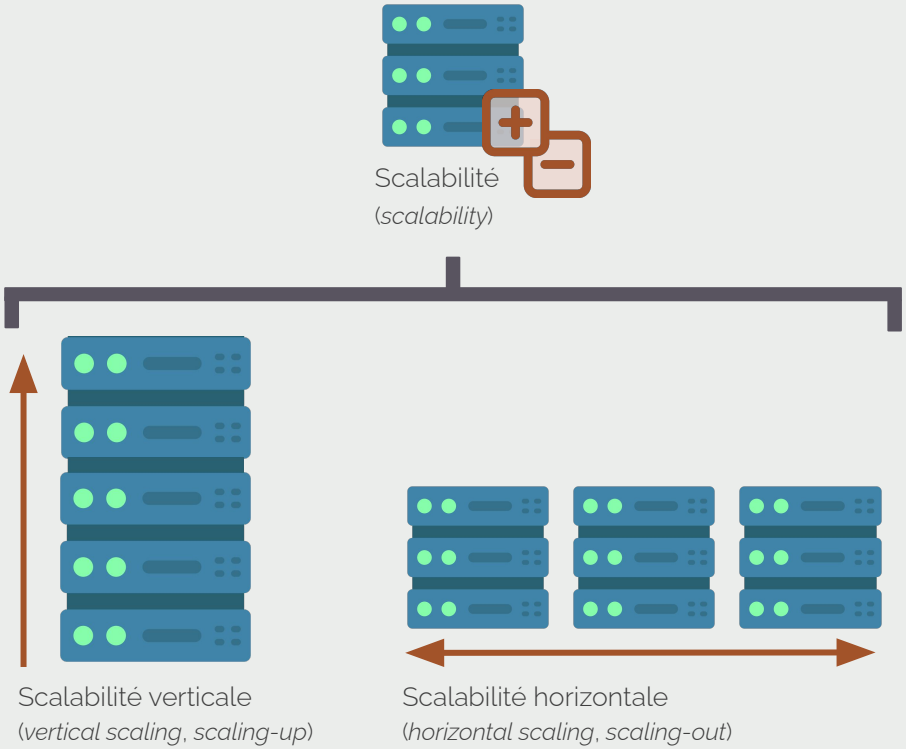
Scalabilité : Capacité d'un système à s'adapter à un changement d'échelle (augmentation ou réduction).



Scalabilité

Scalabilité : Capacité d'un système à s'adapter à un changement d'échelle (augmentation ou réduction).

Scalabilité verticale : Améliorer la capacité des ressources existantes au sein d'un seul serveur (RAM, processeurs, carte réseau, disque dur ...).



Scalabilité

Scalabilité : Capacité d'un système à s'adapter à un changement d'échelle (augmentation ou réduction).

Scalabilité verticale : Améliorer la capacité des ressources existantes au sein d'un seul serveur (RAM, processeurs, carte réseau, disque dur ...).

Scalabilité horizontale : Augmenter la capacité d'un système en ajoutant des serveurs au réseau, appelés *noeuds*.

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

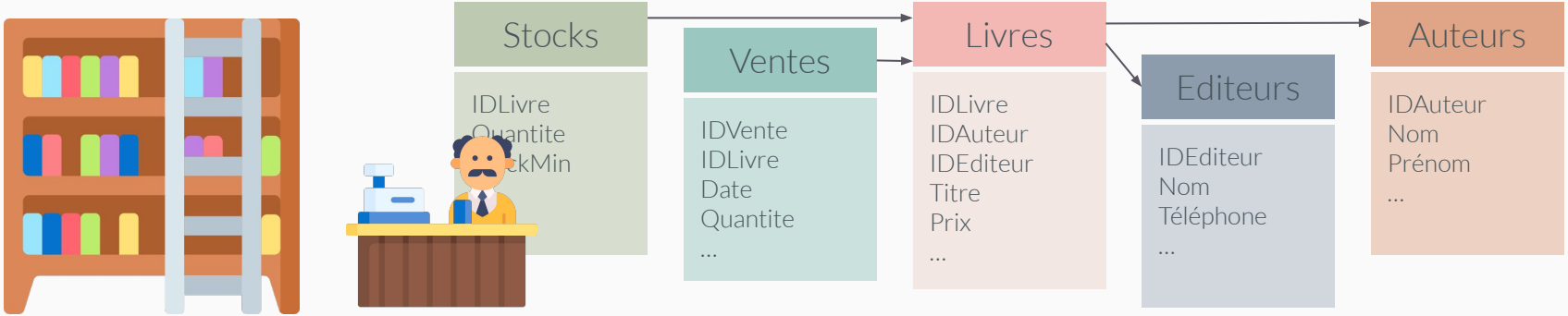
Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès

  Exemple :

  - Trading haute fréquence
  - Voiture autonome

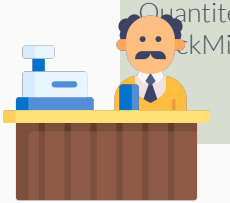
Du tableur à la base de données relationnelle



Et puis, M. Dupont a eu une idée : il veut organiser des **ventes spéciales** tout au long de l'année pour inciter ses clients à revenir régulièrement. Par exemple :

- Pour les vacances d'été, paréos et cartes postales pour lire sur la plage et parler de ses lectures
- Pour les vacances de Noël, tasses et plaids pour des après-midis lecture cocooning

Du tableur à la base de données relationnelle



Stocks
IDLivre
Quantite
ckMin

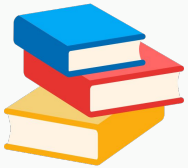
Ventes
IDVente
IDLivre
Date
Quantite
...

Livres
IDLivre
IDAuteur
IDEditeur
Titre
Prix
...

Editeurs
IDEditeur
Nom
Téléphone
...

Auteurs
IDAuteur
Nom
Prénom
...

M. Dupont réalise que sa **base de données** n'est **pas adaptée** pour ces nouveaux produits :

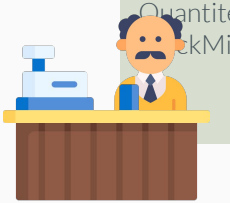


```
titre
auteur
éditeur
ISBN
format
...
```

```
matériau
couleur
contenance
fabriquant
lave vaisselle
...
```

```
matière
poids
longueur
largeur
enveloppe
...
```

Du tableur à la base de données relationnelle



Stocks

?
Quantite
ckMin

Ventes

IDVente
?
Date
Quantite
...

?

?

?

?

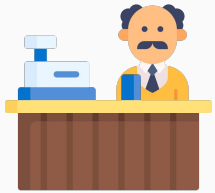
?

?

À **chaque** nouveau type de produit, donc à chaque vente spéciale, le libraire doit, au choix :

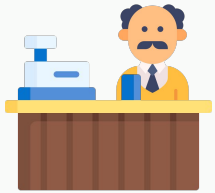
- Ajouter de **nouvelles tables**
- **Modifier les tables existantes** : ajouter des colonnes optionnelles qui resteront vides pour certains produits

Du tableur à la base de données relationnelle



« La base de données relationnelle a apporté efficacité et fiabilité, mais chaque nouveauté, même éphémère, m'oblige à revoir la structure … »

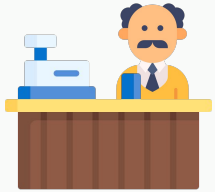
Du tableur à la base de données relationnelle



Pour couronner le tout, M. Dupont aimerait permettre aux clients de laisser des avis et des notes sur les livres qu'ils ont achetés, afin de créer une communauté et guider les lecteurs dans leurs achats (forum). Un avis peut contenir :

- Du **texte libre**
- Une note
- Des **photos**

Du tableur à la base de données relationnelle



Le libraire est alors confronté à une autre limite des bases de données relationnelles : elles ne sont **pas adaptées** pour stocker des **données non structurées**, comme des milliers de textes très longs ou des images.

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

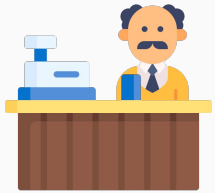
Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès
- **Flexibilité** : évolution des besoins, variété des données (structurées, semi-structurées, non structurées)

Du tableur à la base de données relationnelle



Avantages :

- Stockage structuré, efficace et fiable
- Langage de requête SQL
- Adapté aux gros volumes de données structurées

Inconvénients :

- Manque de flexibilité
- Non adaptées aux données massives
- Non adaptées aux données non structurées
- Limitations vitesse de traitement des requêtes

« Le site a du succès … mais la base de données devient un frein. »

ATTENTION !

- La question n'est pas "Quelle est la meilleure solution *dans l'absolu* ?" mais "Quelle est la solution **la plus adaptée à mes besoins** ?"
- Chaque solution vient avec des avantages et des inconvénients
- Les bases de données NoSQL **n'ont pas vocation à remplacer** les bases de données relationnelles
- La scalabilité horizontale, la haute disponibilité et les performances élevées (pour certaines requêtes) ont plusieurs inconvénients, dont certaines limitations quant à l'intégrité des données, et réciproquement (voir théorème CAP)
- Exemple : Pour les banques et toutes les transactions financières, les bases de données relationnelles restent un incontournable

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès
- **Flexibilité** : évolution des besoins, variété des données (structurées, semi-structurées, non structurées)

Du relationnel au NoSQL

À partir de 2000 : Nouveaux besoins, Nouvelles opportunités

Contexte :

- Essor du **Web** et de son **nombre d'utilisateurs**
- Avènement du **Big Data** (volume, variété, vélocité)

Besoins :

- **Scalabilité** : volume et vélocité des données (stockage), demande d'accès croissant
- **Disponibilité** : accès continu, sans interruption
- **Performance** : vélocité des données (traitement), rapidité d'accès
- **Flexibilité** : évolution des besoins, variété des données (structurées, semi-structurées, non structurées)

Opportunité :

- **Commodity hardware** (matériel standard) : **cluster**

Du relationnel au NoSQL

Commodity hardware (matériel standard) :

- Matériel non spécialisé et non propriétaire, donc facilement interchangeable avec d'autres matériels courants
- Matériel produit en masse, donc largement disponible
- Matériel peu coûteux

Exemple : PC classique, disque dur standard, serveur générique

Les commodity hardware facilitent la scalabilité horizontale, notamment grâce à la réduction des coûts

Cluster : Ensemble de serveurs interconnectés fonctionnant comme un seul système

Du relationnel au NoSQL

Avènement du NoSQL

Les entreprises du Web confrontées à ces difficultés sont les premières à développer en interne des solutions sur mesure

- BigTable (Google, 2005)
- DynamoDB (Amazon, 2007)
- Cassandra (Facebook, 2008)

Du relationnel au NoSQL

Avènement du NoSQL

Les entreprises du Web confrontées à ces difficultés sont les premières à développer en interne des solutions sur mesure

- BigTable (Google, 2005)
- DynamoDB (Amazon, 2007)
- Cassandra (Facebook, 2008)

Popularisation du NoSQL

- Multiplication des initiatives open source
- Popularisation du terme “**NoSQL**” en 2009

Chapitre 1. Du modèle relationnel au NoSQL

Chapitre 2. NoSQL : les concepts

Chapitre 3. NoSQL : les catégories

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
    - Non relationnel
    - En majorité open-source

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

- La **structure** de la base de données **doit** être **définie à l'avance**, avant toute insertion de données

```
CREATE TABLE utilisateur
(
  id INT PRIMARY KEY NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255),
  date_naissance DATE
)
```

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

- La **structure** de la base de données **doit** être **définie à l'avance**, avant toute insertion de données
- Toute donnée doit être **strictement** conforme au format prédéfini pour être intégrée à la base

```
CREATE TABLE utilisateur
(
  id INT PRIMARY KEY NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255),
  date_naissance DATE
)
```

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

- La **structure** de la base de données **doit** être **définie à l'avance**, avant toute insertion de données
- Toute donnée doit être **strictement** conforme au format prédéfini pour être intégrée à la base

```
CREATE TABLE utilisateur
(
  id INT PRIMARY KEY NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255),
  date_naissance DATE
)
```

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

- Il n'est **pas nécessaire** de définir la structure de la base de données avant l'insertion de données

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

- La **structure** de la base de données **doit** être **définie à l'avance**, avant toute insertion de données
- Toute donnée doit être **strictement** conforme au format prédéfini pour être intégrée à la base

```
CREATE TABLE utilisateur
(
  id INT PRIMARY KEY NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255),
  date_naissance DATE
)
```

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

- Il n'est **pas nécessaire** de définir la structure de la base de données avant l'insertion de données
- La **structure existe**, même **implicitement**, mais elle peut être **modifiée à la volée**

```
{
  prenom : "Mary",
  nom : "Poppins",
  pseudo : "dernier_chat_pitre"
  games :
   [ { game : "G-6192", opponent : "choucroute" },
     { game : "G-1004", opponent : "yuccai" } ]
}
```

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

- La **structure** de la base de données **doit** être **définie à l'avance**, avant toute insertion de données
- Toute donnée doit être **strictement** conforme au format prédéfini pour être intégrée à la base

```
CREATE TABLE utilisateur
(
  id INT PRIMARY KEY NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255),
  date_naissance DATE
)
```

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

- Il n'est **pas nécessaire** de définir la structure de la base de données avant l'insertion de données
- La **structure existe**, même **implicitement**, mais elle peut être **modifiée à la volée**
- Il est **possible** d'intégrer progressivement des contraintes (i.e un schéma)

```
{
  prenom : "Mary",
  nom : "Poppins",
  pseudo : "dernier_chat_pitre"
  games :
   [ { game : "G-6192", opponent : "choucroute" },
     { game : "G-1004", opponent : "yuccai" } ]
}
```

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Toute modification requiert des **migrations parfois complexes**
- **Données hétérogènes**
- **Données non structurées**

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- **Modifications simples et rapides**
- **Données hétérogènes**
- **Données non structurées**

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite**

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Toute modification requiert des **migrations parfois complexes**
- **Données hétérogènes**
- **Données non structurées**

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- **Modifications simples et rapides**
- **Données hétérogènes**
- **Données non structurées**

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite** (frein à la prise en main de la base de données)

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Toute modification requiert des **migrations parfois complexes**
- **Données hétérogènes**
- **Données non structurées**

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- **Modifications simples et rapides**
- **Données hétérogènes**
- **Données non structurées**

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite**

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Certaines modifications requièrent des **migrations complexes**
- Données hétérogènes
- Données non structurées

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- Modifications simples et rapides
- Données hétérogènes
- Données non structurées

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite**

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Certaines modifications requièrent des **migrations complexes**
- Données hétérogènes
- Données non structurées

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- **Modifications simples et rapides**
- Données hétérogènes
- Données non structurées

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite**

Base de données relationnelle

Schéma rigide (*fixed schema*, *static schema*)

Point fort :

- **Intégrité des données** garantie par les procédures de contrôle
- **Structure** des données **explicite**

Point faible :

- Mise en place préalable (**design-then-code**)
- Certaines modifications requièrent des **migrations complexes**
- **Données hétérogènes**
- Données non structurées

Base de données NoSQL

Schéma flexible (*schema-optional*, *schema-less*, *dynamic schema*)

Point fort :

- Pas de mise en place préalable (**code-first**)
- **Modifications simples et rapides**
- **Données hétérogènes**
- Données non structurées

Point faible :

- En l'absence de contraintes, **risque** de **corruption silencieuse**, d'**inexactitude**, d'**incomplétude**, et d'**incohérence** des données
- **Structure** des données **implicite**

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application

Orientée application : La **structure** de stockage des données est **alignée sur les besoins d'accès de l'application** : les données fréquemment lues ou modifiées ensemble sont regroupées.

Autrement dit : les données sont stockées en fonction de l'utilisation que l'on souhaite faire de la base de données

Orientée application : La **structure** de stockage des données est **alignée sur les besoins d'accès de l'application** : les données fréquemment lues ou modifiées ensemble sont regroupées.

Avantage :

- **Rapidité d'accès** pour les **requêtes** qui **correspondent aux cas d'usage** considérés lors de la création de la base de données

Inconvénient :

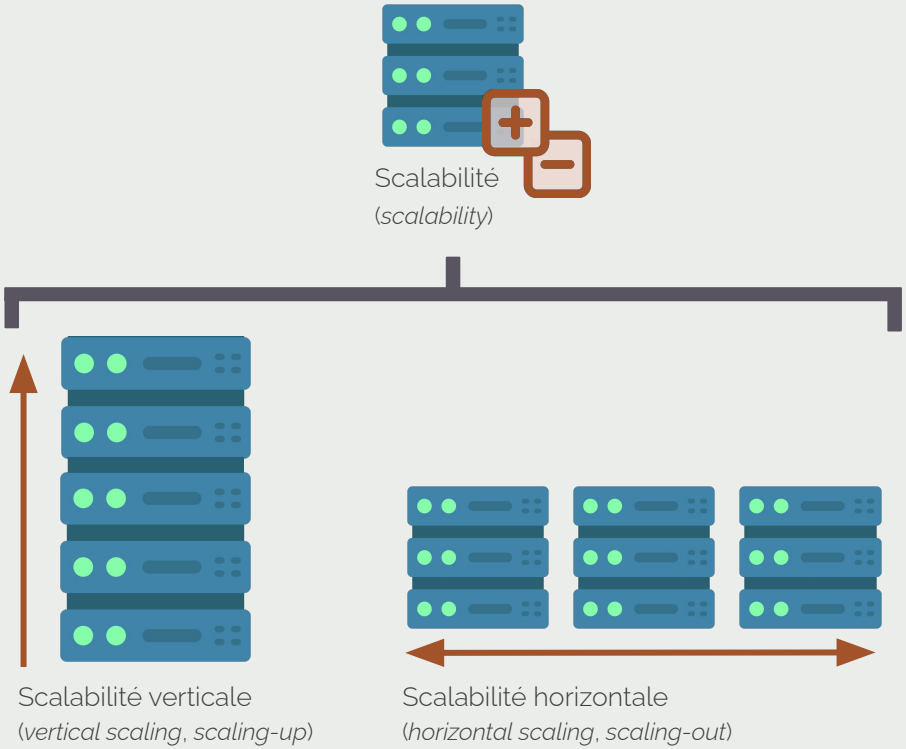
- **Performances variables** pour les **requêtes** qui **ne correspondent pas aux cas d'usage** considérés lors de la création de la base de données

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale



Scalabilité

Scalabilité : Capacité d'un système à s'adapter à un changement d'échelle (augmentation ou réduction).

Scalabilité horizontale : Augmenter la capacité d'un système en ajoutant des serveurs au réseau, appelés *noeuds*.

Scalabilité verticale : Améliorer la capacité des ressources existantes au sein d'un seul serveur (RAM, processeurs, carte réseau, disque dur ...).

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale

Comment cela se traduit-il en pratique ?

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale

Comment cela se traduit-il en pratique ?

À quoi doit-on cette scalabilité horizontale ?

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale

Comment cela se traduit-il en pratique ?

À quoi doit-on cette scalabilité horizontale ?

Concept d’agrégat

Base de données NoSQL : les concepts

- NoSQL : **“Not only SQL”** (rétro-acronyme), “non relationnelle”
- Grande variété de solutions
- Points communs :
  - Non relationnel
  - En majorité open-source
  - Schéma flexible
  - Orienté application
  - Scalabilité horizontale
  - En majorité orienté agrégats

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité** cohérente et indissociable.

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité** cohérente et indissociable.

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité** cohérente et indissociable.

lien = le plat que l'on souhaite cuisiner

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité** cohérente et indissociable.

```
{
  nom : Waterzoï de volaille,

  ingrédients : [poireau, carotte,
  branche de céleri, cuisse de poulet,
  vin blanc, crème fraîche]

  recette : {
              étape 1 : ...,
              étape 2 : ...,
              ...
            }

  ustensiles : [faitout, couteau, épluche
  légume, cuillère en bois, spatule]
}
```

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité** cohérente et indissociable.

Un agrégat est un bloc qui se suffit à lui-même : en une seule requête, on obtient toutes les informations dont on a besoin pour cuisiner du Waterzoï de volaille.

```
{
  nom : Waterzoï de volaille,

  ingrédients : [poireau, carotte, branche de céleri, cuisse de poulet, vin blanc, crème fraîche]

  recette : {
              étape 1 : ...,
              étape 2 : ...,
              ...
            }

  ustensiles : [faitout, couteau, épluche légume, cuillère en bois, spatule]
}
```

Avec une base de données relationnelle :

Ingrédient
ID_ingredient nom ...

Recette
ID_recette nom instructions ...

Recette_Ingrédient
ID_recette ID_ingrédient Mesure_quantité Mesure_unité ...

...
...

Les informations pour cuisiner un Waterzoï de volaille sont réparties dans plusieurs tables. Il faut faire plusieurs jointures **coûteuses** pour les rassembler.

Avec une base de données relationnelle :

Ingrédient
ID_ingredient nom ...

Recette
ID_recette nom instructions ...

Recette_Ingrédient
ID_recette ID_ingrédient Mesure_quantité Mesure_unité ...

...
...

Agrégat : Ensemble d'**objets** étroitement liés traités comme une **unité cohérente et indissociable**.

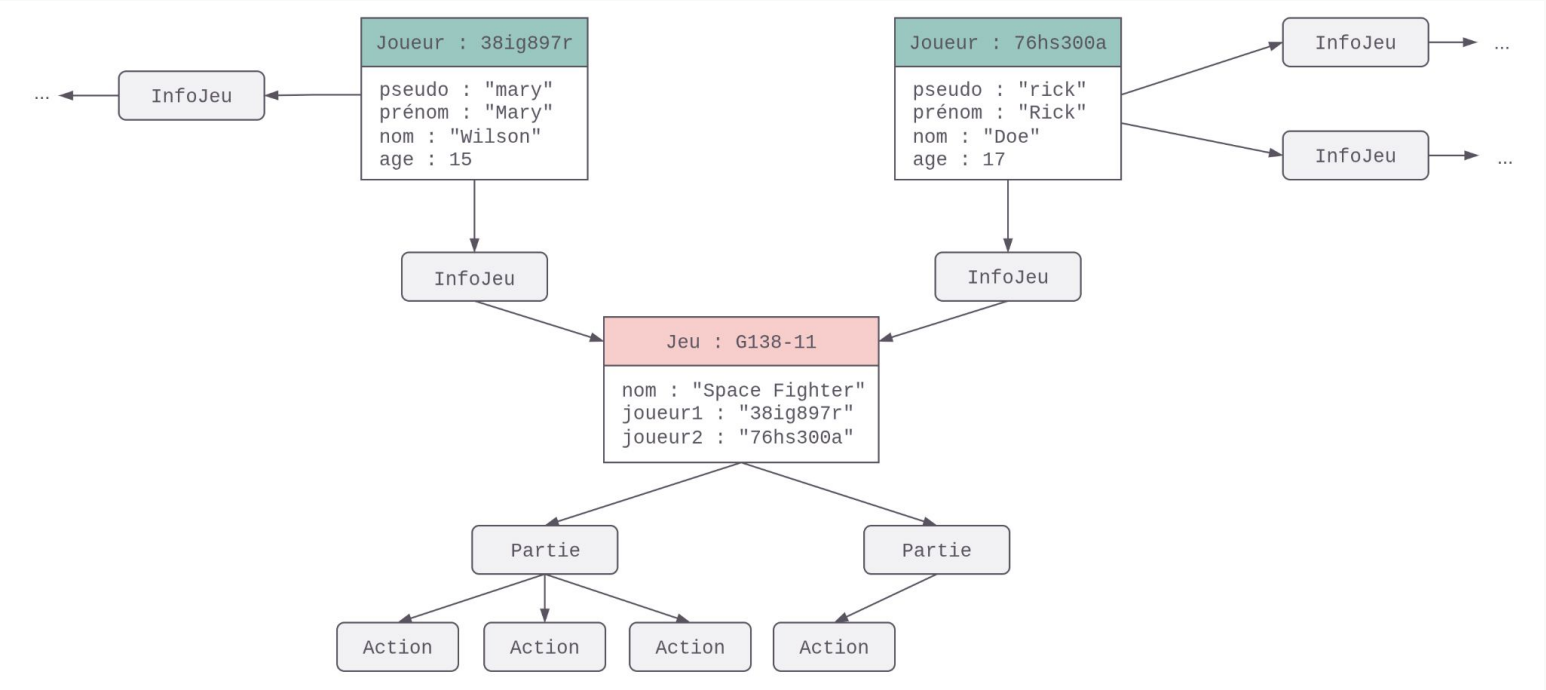
Base de données NoSQL **orientée agrégats** :

- Les données **souvent** consultées ou modifiées ensemble sont regroupées au sein d'un **même agrégat**

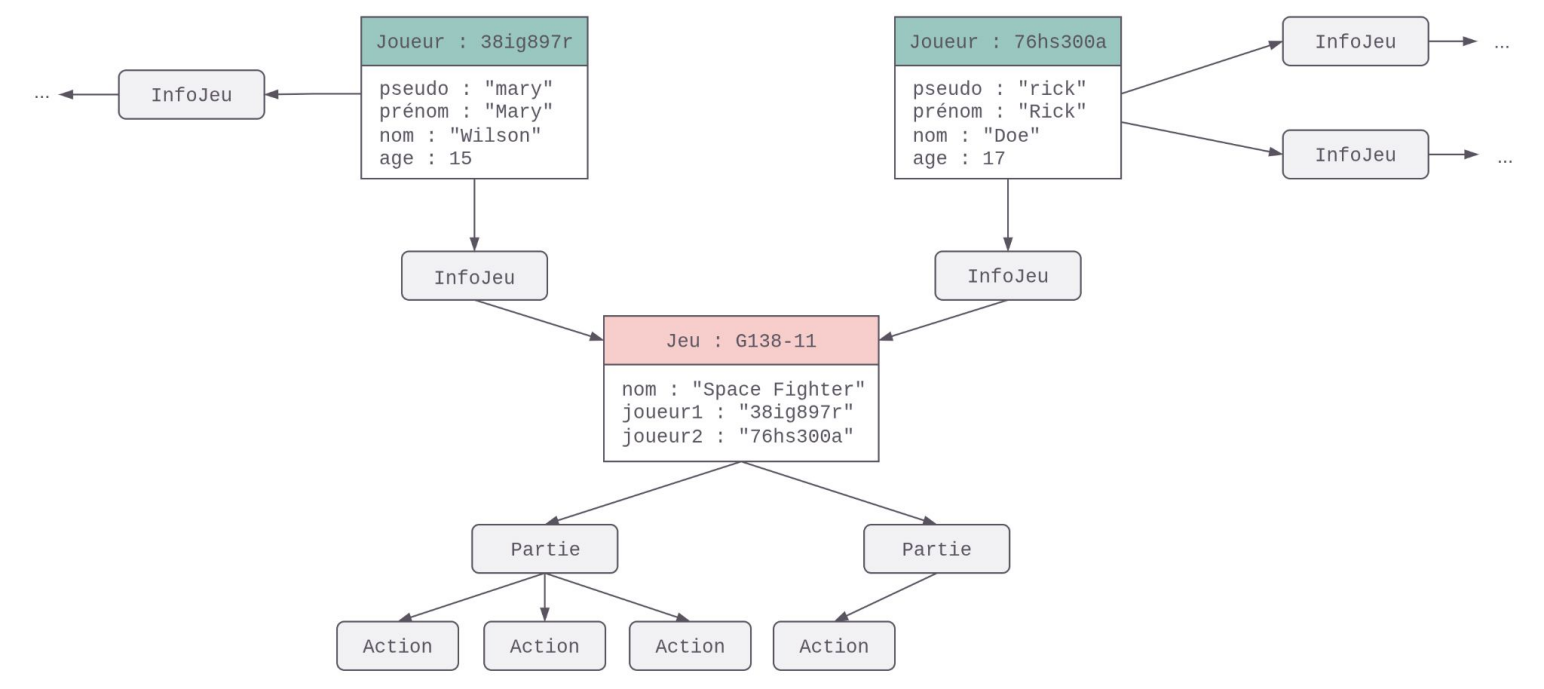
Agrégat : Ensemble d'**objets** étroitement liés traités comme une **unité cohérente et indissociable**.

Base de données NoSQL **orientée agrégats** :

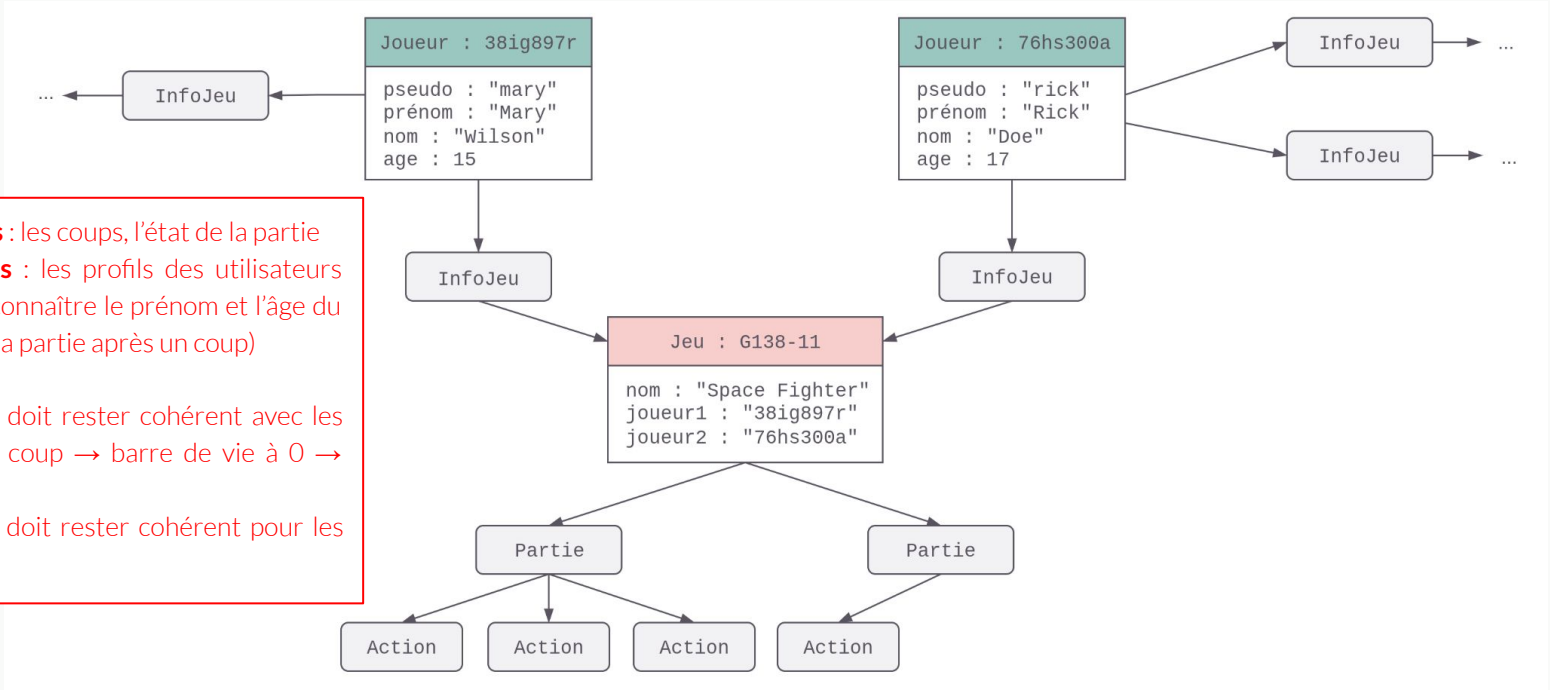
- Les données **souvent** consultées ou modifiées ensemble sont regroupées au sein d'un **même agrégat**
- Plusieurs représentations des données sont possibles



- Comment est-ce que je compte utiliser ma base de données ?
- De quelles informations ai-je le plus fréquemment besoin ? Et inversement : quelles informations ne sont pas critiques ? Les agrégats ne doivent être ni trop petits ni trop gros
- Quelles règles doivent absolument être respectées ? Ces règles doivent pouvoir être contrôlées au sein de l'agrégat



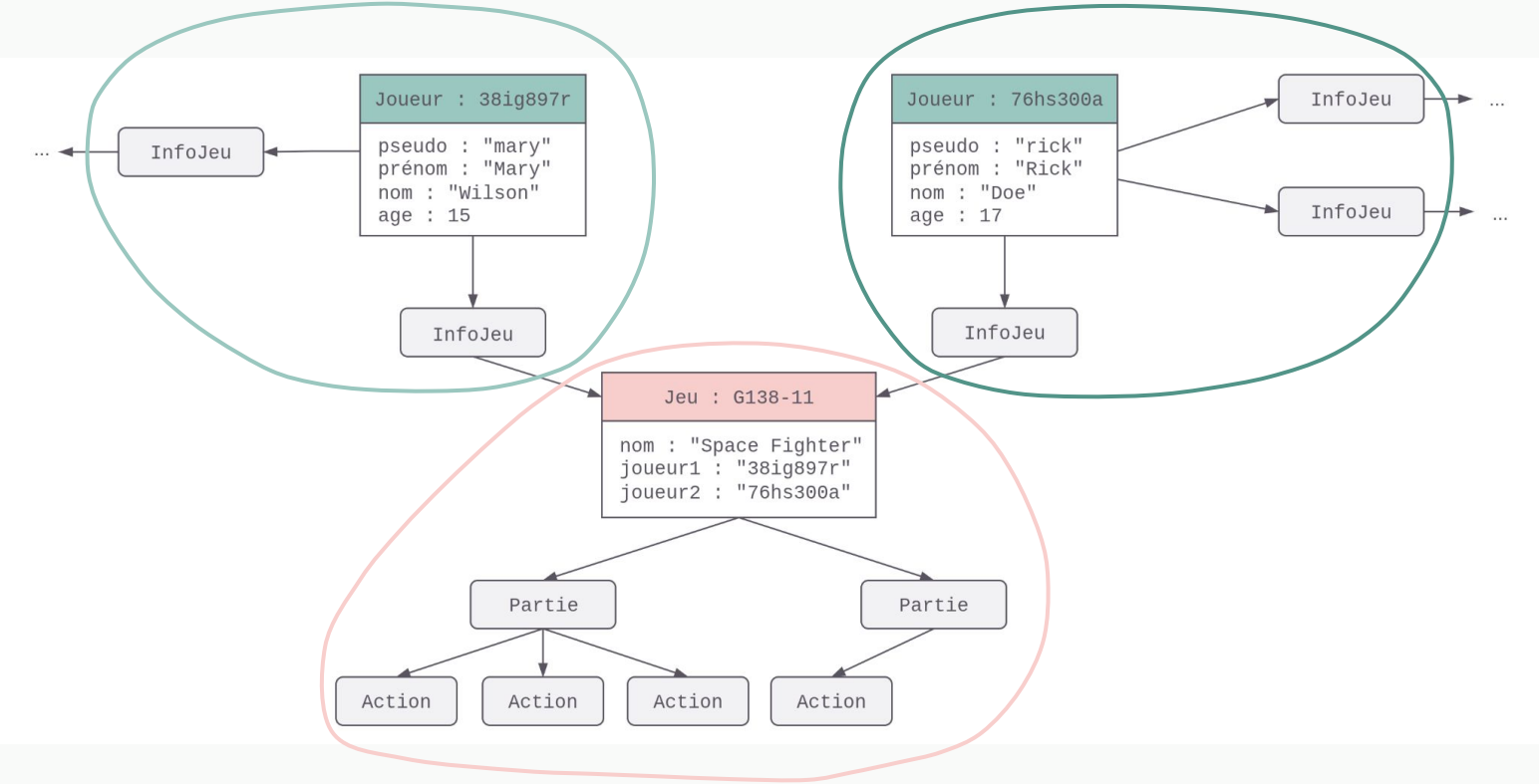
Application : Gestionnaire de parties
Racine de l'agrégat : ?



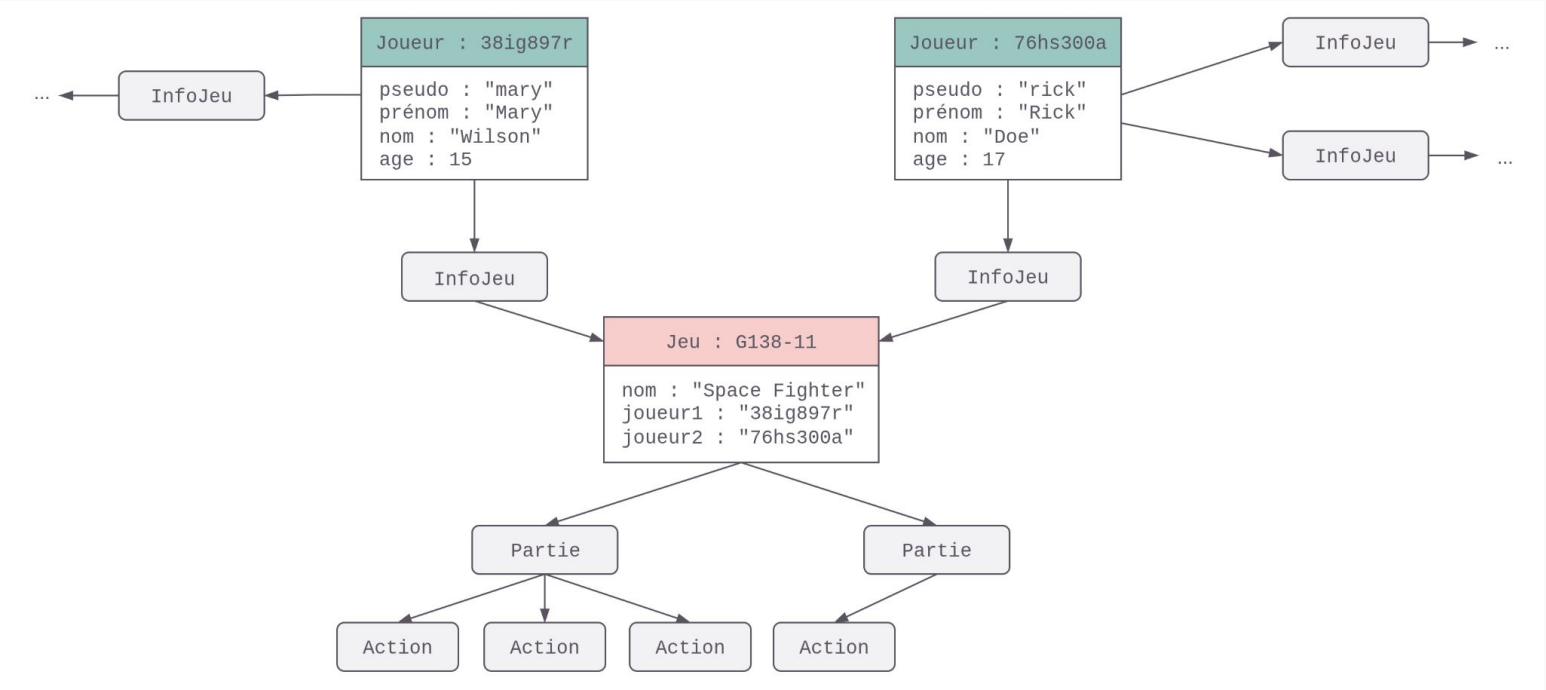
Informations essentielles : les coups, l'état de la partie
Informations secondaires : les profils des utilisateurs (ex : je n'ai pas besoin de connaître le prénom et l'âge du joueur pour mettre à jour la partie après un coup)
Règles à respecter :

- l'état de la partie doit rester cohérent avec les coups joués (ex : coup → barre de vie à 0 → partie terminée)
- l'état de la partie doit rester cohérent pour les deux joueurs

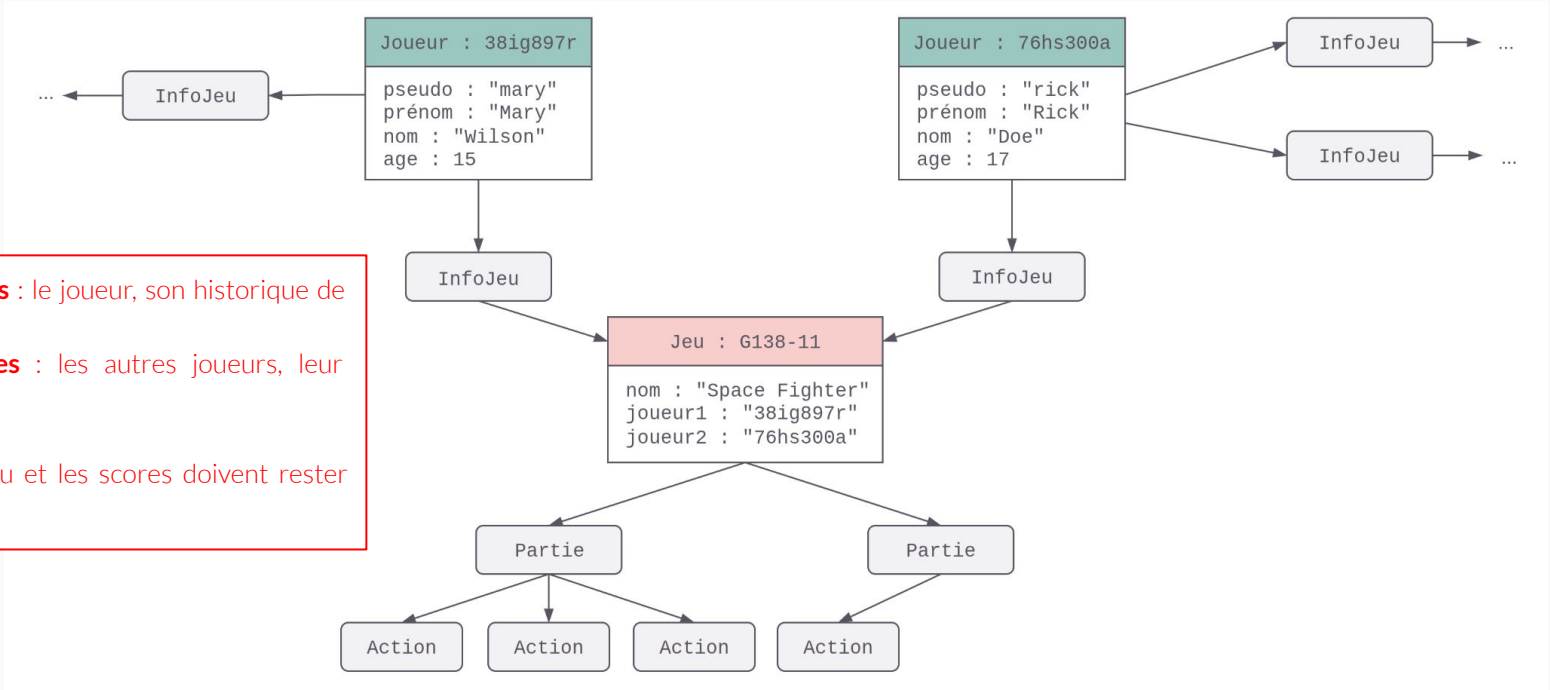
Application : Gestionnaire de parties
Racine de l'agrégat : ?



Application : Gestionnaire de parties
Racine de l'agrégat : `Jeu`



Application : Gestionnaire du profil, de l'historique et des scores des joueurs
Racine de l'agrégat : ?



Informations essentielles : le joueur, son historique de jeu, ses scores

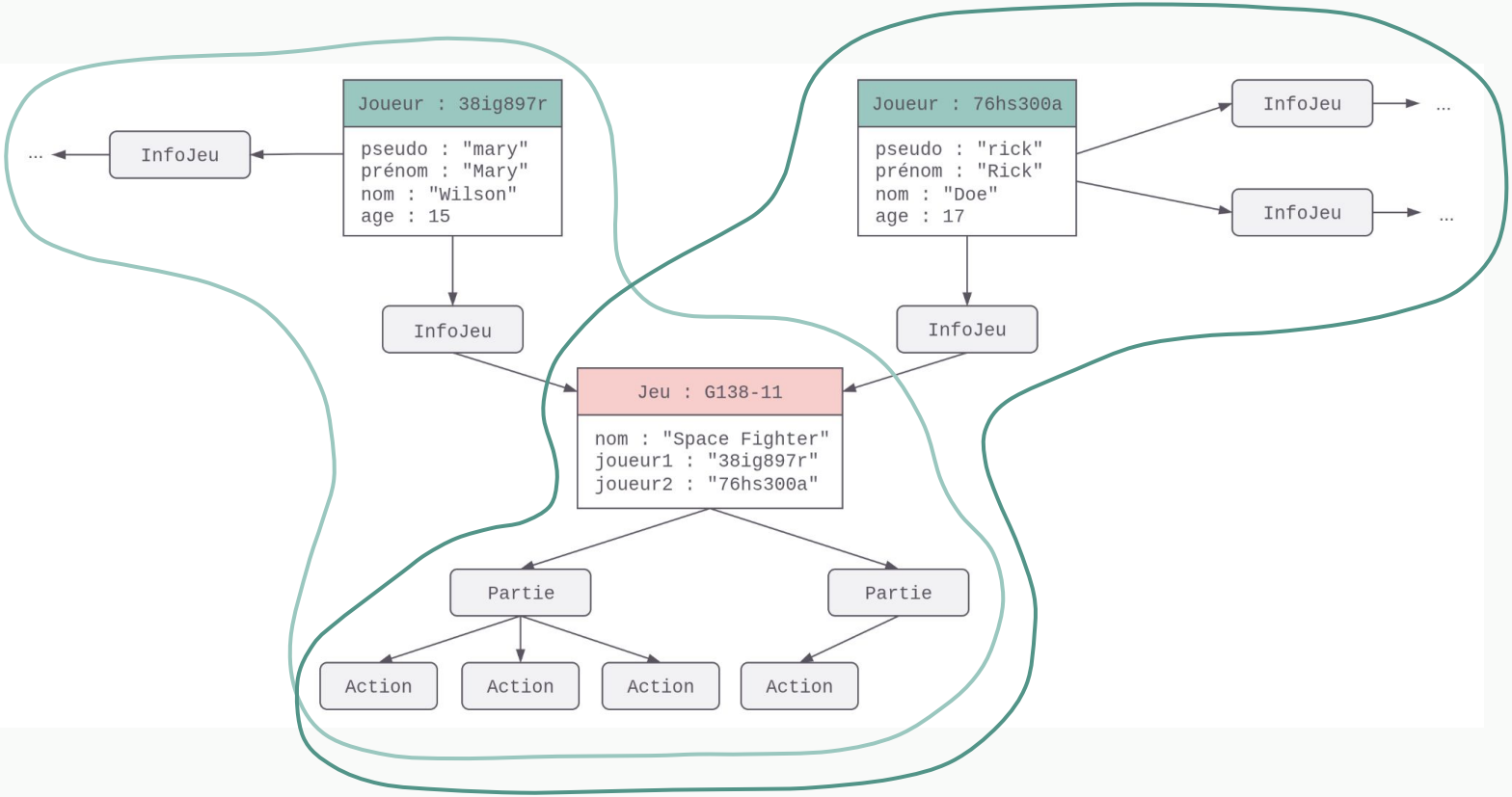
Informations secondaires : les autres joueurs, leur historique, leurs scores

Règles à respecter :

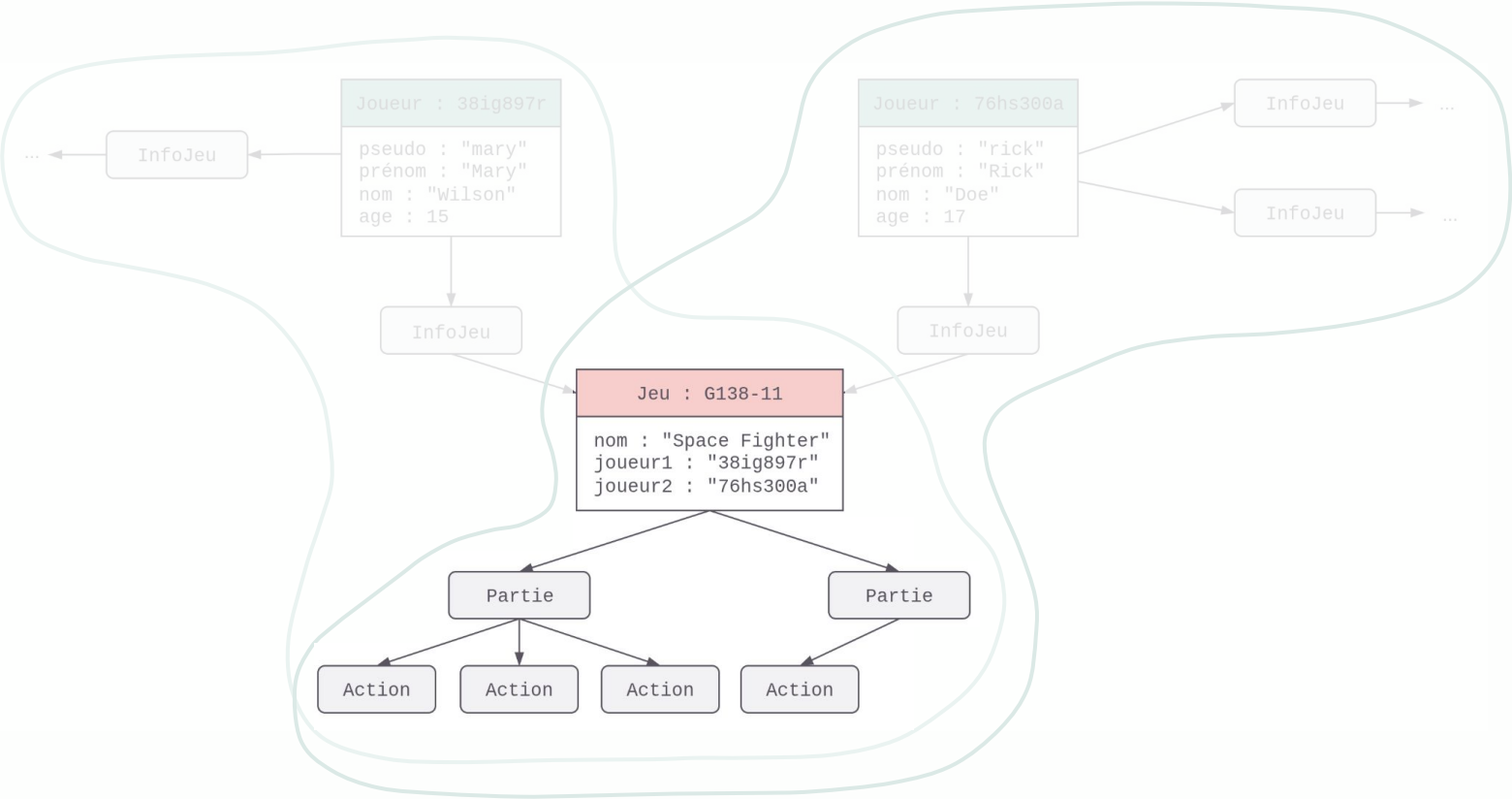
- l'historique de jeu et les scores doivent rester cohérents

Application : Gestionnaire du profil, de l'historique et des scores des joueurs

Racine de l'agrégat : ?

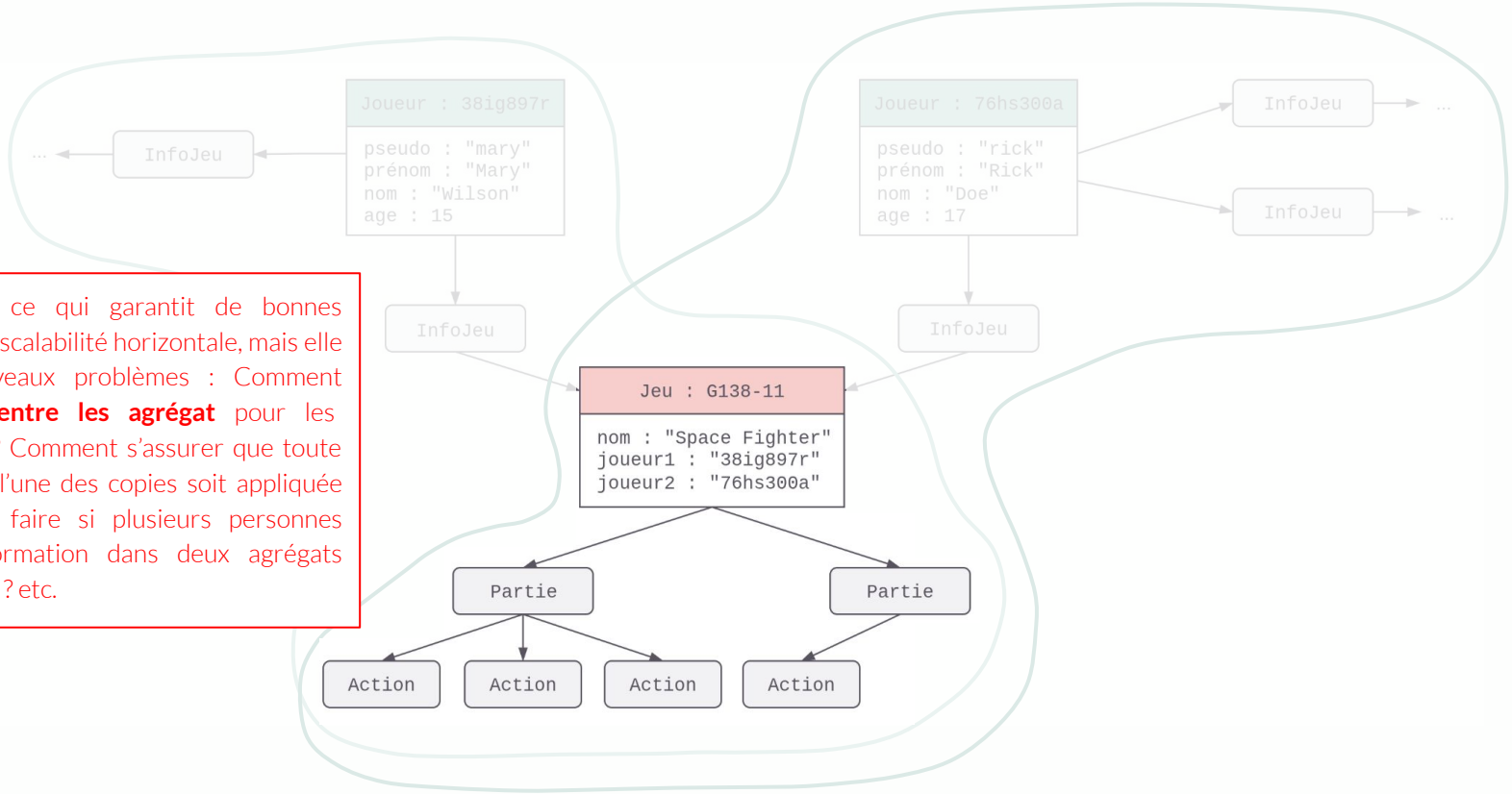


Application : Gestionnaire du profil, de l'historique et des scores des joueurs
Racine de l'agrégat : `Joueur`



Remarque : Duplication de certaines informations (dénormalisation)

La dénormalisation est ce qui garantit de bonnes performances ainsi que la scalabilité horizontale, mais elle introduit aussi de nouveaux problèmes : Comment garantir la **cohérence entre les agrégat** pour les informations dupliquées ? Comment s'assurer que toute modification réalisée sur l'une des copies soit appliquée aux autres ? Comment faire si plusieurs personnes modifient la même information dans deux agrégats différents en même temps ? etc.



Remarque : Duplication de certaines informations (dénormalisation)

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité cohérente et indissociable**.

Base de données NoSQL **orientée agrégats** :

- Les données **souvent consultées ou modifiées ensemble** sont regroupées au sein d'un **même agrégat**
- Plusieurs représentations des données sont possibles

Agrégat : Ensemble d'**objets** étroitement **liés** traités comme une **unité cohérente et indissociable**.

Base de données NoSQL **orientée agrégats** :

- Les données **souvent consultées ou modifiées ensemble** sont regroupées au sein d'un **même agrégat**
- Plusieurs représentations des données sont possibles
- Les **frontières des agrégats** sont choisies **en fonction des cas d'usage**
  D'où : "orientées application"

Avantage :

- **Rapidité d’accès** pour les requêtes *en fonction* desquelles les agrégats ont été construits (pas de jointures)
- **Scalabilité horizontale** : agrégats autosuffisants facilement distribuables sur un cluster car peu d'interactions entre agrégats
- **Forte disponibilité** : disponibilité permise par la scalabilité horizontale (sharding, réplication)

Avantage :

- **Rapidité d'accès** pour les requêtes *en fonction* desquelles les agrégats ont été construits (pas de jointures)
- **Scalabilité horizontale** : agrégats autosuffisants facilement distribuables sur un cluster car peu d'interactions entre agrégats
- **Forte disponibilité** : disponibilité permise par la scalabilité horizontale (sharding, réplication)

Inconvénients :

- **Intégrité des données** plus difficile à garantir : **dénormalisation**
- **Performances variables** pour toutes **requêtes autres** que **celles** envisagées **à la création des agrégats**

Chapitre 1. Du modèle relationnel au NoSQL

Chapitre 2. NoSQL : les concepts

Chapitre 3. NoSQL : les catégories

Base de données NoSQL : les catégories

Il existe quatre catégories de bases de données NoSQL :

- **Base de données clé-valeur**
  Key-value database
- **Base de données orientées documents**
  Document-oriented database
- **Base de données en familles de colonnes**
  Column-family database, Wide-column database
- **Base de données de graphe**
  Graph database

Base de données NoSQL : les catégories

Il existe quatre catégories de bases de données NoSQL :

- **Base de données clé-valeur**
  Key-value database
- **Base de données orientées documents**
  Document-oriented database
- **Base de données en familles de colonnes**
  Column-family database, Wide-column database
- **Base de données de graphe**
  Graph database

Base de données clé-valeur

- Base de données NoSQL la plus simple

Base de données clé-valeur

- Base de données NoSQL la plus simple
- Collection de **paires clé-valeurs** où :
  - Les **clés** sont **uniques**
  - Les **valeurs** sont des **agrégats opaques** (BLOB) :

    Sauf exception, les valeurs ne sont **pas directement interrogeables**. Il faut charger en mémoire l'intégralité de la valeur à l'aide de la clé pour accéder à son contenu. Toute modification implique une **réécriture complète**.

Base de données clé-valeur

- Base de données NoSQL la plus simple
- Collection de **paires clé-valeurs** où :
  - Les **clés** sont **uniques**
  - Les **valeurs** sont des **agrégats opaques** (BLOB)
- Requêtes via les clés uniquement

Base de données clé-valeur

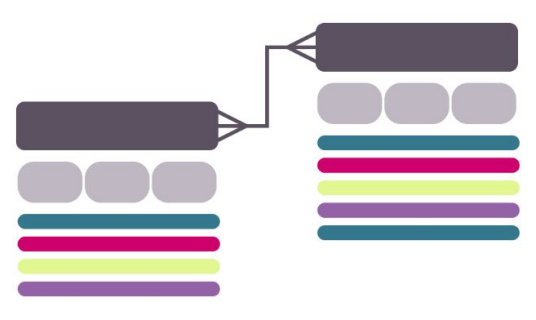
- Base de données NoSQL la plus simple
- Collection de **paires clé-valeurs** où :
  - Les **clés** sont **uniques**
  - Les **valeurs** sont des **agrégats opaques** (BLOB)
- Requêtes via les clés uniquement
- **Accès ultra-rapide** aux données (table de hachage)

Base de données clé-valeur

Clé	Valeur
/Cart/user17501/-/currency	EUR
/Cart/user17501/-/total	35.70
/Cart/user17501/-/updatedAt	2025-09-09T12:34:56Z
/Cart/user17501/-/items/0/productId	P42-Z88
/Cart/user17501/-/items/0/qty	2
/Cart/user17501/-/items/0/price	12.90
...	...

panier d'achat : stockage temporaire des informations d'achat sur le site de M. Dupont

Clé	Valeur
Cart:user17501	"{"currency":"EUR","total":"35.7", "updatedAt":"2025-09-09T12:34:56Z", "items":"[{"productId":"P42-Z88","qty":2,"price":12.90}, ...}"



Base de données relationnelles

Base de données clé-valeur

Base de données NoSQL : les catégories

Il existe quatre catégories de bases de données NoSQL :

- **Base de données clé-valeur**
  Key-value database
- **Base de données orientées documents**
  Document-oriented database
- **Base de données en familles de colonnes**
  Column-family database, Wide-column database
- **Base de données de graphe**
  Graph database

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**
  - Les **valeurs** sont **consultables** :

    Il est possible d'interroger le contenu d'un document et de le modifier—totalement ou partiellement—sans avoir besoin de récupérer l'intégralité du document à l'aide de sa clé.

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**
  - Les **valeurs** sont **consultables**
  - Les **documents** peuvent présenter des **similarités** sur le plan structurel et/ou conceptuel mais ne sont **pas identiques**

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**
  - Les **valeurs** sont **consultables**
  - Les **documents** peuvent présenter des **similarités** sur le plan structurel et/ou conceptuel mais ne sont **pas identiques**
  - Les documents peuvent être au format XML, JSON, BSON …

Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**
  - Les **valeurs** sont **consultables**
  - Les **documents** peuvent présenter des **similarités** sur le plan structurel et/ou conceptuel mais ne sont **pas identiques**
  - Les documents peuvent être au format XML, JSON, BSON …
- **Requêtes** via les **valeurs** des documents

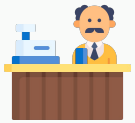
Base de données orientées documents

- Base de données NoSQL la plus largement utilisée
- Collection de **documents** où :
  - Un document est un ensemble de **paires clé-valeurs**
  - Les **valeurs** sont **consultables**
  - Les **documents** peuvent présenter des **similarités** sur le plan structurel et/ou conceptuel mais ne sont **pas identiques**
  - Les documents peuvent être au format XML, JSON, BSON …
- **Requêtes** via les **valeurs** des documents
- **Grande flexibilité** : **données hétérogènes**, données **non structurées**, facilité de **modification**

Base de données orientées documents

```
{
  title : "Un livre incontournable",
  content : "L'un des meilleurs romans policiers d'Agatha Christie, avec l'incontournable Hercule Poirot ! Bien construit, sans hémoglobine, il propose une réflexion sur la fiabilité des témoignages lorsqu'on est en quête de vérité.",
  author : {
            pseudo : "LeBuveurDencre",
            email : "ink.drinker@email.com"
          },
  creation_date : 2023-09-20,
  book : "BK-12_0947"
  tags : ["policier","psychologique"],
  comment: [{
              content : "Tout à fait d'accord ! Il fait partie de mon top 3 des romans d'Agatha Christie !",
              author : "Bubulle"
              creation_date : "2023-09-20"
            }]
  like: 4
}
```

forum de M. Dupont



Base de données orientées documents

```
{
  title : "Un livre incontournable",
  content : "L'un des meilleurs romans policiers d'Agatha Christie, avec l'incontournable Hercule Poirot ! Bien construit, sans hémoglobine, il propose une réflexion sur la fiabilité des témoignages lorsqu'on est en quête de vérité.",
  author : {
            pseudo : "LeBuveurDencre",
            email : "ink.drinker@email.com"
          },
  creation_date : 2023-09-20,
  book : "BK-12_0947"
  tags : ["policier","psychologique"],
  comment: [{
             content : "Tout à fait d'accord ! Il fait partie de mon top 3 des romans d'Agatha Christie !",
             author : "Bubulle"
             creation_date : "2023-09-20"
           }]
  like: 4
}
```

```
{
  title : "Un livre qui résonne comme une alerte",
  content : "Certains livres ne se contentent pas d'éclairer le passé : ils dissèquent aussi les mécanismes du présent. "Les Irresponsables: Qui a porté Hitler au pouvoir ?" de Johann Chapoutot est de ceux-là. [...]",
  author : {
            pseudo : "PostTenebrasLux",
            email : "post.tenebras@email.com"
            bio : "Une lectrice engagée"
          },
  creation_date : 2025-02-26,
  update_date : 2025-02-27,
  book : "BK-03_6533"
  tags : ["historique","nazisme"],
  like: 53
}
```

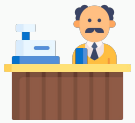
Base de données orientées documents

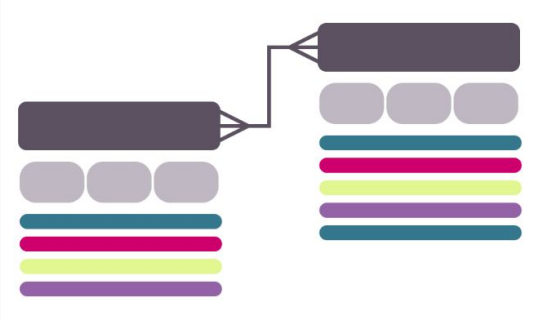
ventes spéciales de M. Dupont

```
{
  category : "book"
  title : "Les Irresponsables: Qui a porté Hitler
  au pouvoir ?",
  author : "Johann Chapoutot",
  publisher : "Éditions Gallimard",
  date : 2025-02-06,
  ISBN : 9782073061195,
  ...
}
```

```
{
  category : "postcard",
  weight : 280,
  weight_unit : "g",
  measurements : "10.5 x 14.8 cm",
  envelope : true
  ...
}
```

```
{
  category : "mug",
  measurements : "13.4 x 10 x 9.8 cm",
  color : ["pink", "red", "green"],
  capacity : 400,
  capacity_unit : "ml"
  manufactured_by : "Zebra A/S",
  dishwasher : true,
  microwave : true,
  oven : true,
  freezer : true
  ...
}
```





Base de données relationnelles

Base de données clé-valeur

Base de données orientées documents

Base de données NoSQL : les catégories

Il existe quatre catégories de bases de données NoSQL :

- **Base de données clé-valeur**
  Key-value database
- **Base de données orientées documents**
  Document-oriented database
- **Base de données en familles de colonnes**
  Column-family database, *Wide-column database*
- **Base de données de graphe**
  Graph database

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**
  - Au sein d'une famille de colonnes, les **lignes** peuvent avoir des **colonnes différentes** :

    Les lignes peuvent partager toutes les colonnes, un sous-ensemble de colonnes ou aucune colonne (ex : absence d'une colonne plutôt que valeur manquante). Une colonne peut être ajoutée à certaines lignes et pas à d'autres.

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**
  - Au sein d'une famille de colonnes, les **lignes** peuvent avoir des **colonnes différentes**
  - Chaque **ligne** est **identifiée par une clé unique** et peut être répartie à travers plusieurs familles de colonnes

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**
  - Au sein d'une famille de colonnes, les **lignes** peuvent avoir des **colonnes différentes**
  - Chaque **ligne** est **identifiée par une clé unique** et peut être répartie à travers plusieurs familles de colonnes
  - Les colonnes d'une même famille sont **stockées de manière contiguë sur le disque**, **ligne par ligne**

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**
  - Au sein d'une famille de colonnes, les **lignes** peuvent avoir des **colonnes différentes**
  - Chaque **ligne** est **identifiée par une clé unique** et peut être répartie à travers plusieurs familles de colonnes
  - Les colonnes d'une même famille sont **stockées de manière contiguë sur le disque**, **ligne par ligne**
- **Requêtes** via les **clés de ligne**, et **parfois** via les **valeurs** de certaines colonnes

Base de données en familles de colonnes

- Données organisées en **familles de colonnes** :
  - Chaque famille de colonnes contient un **ensemble de colonnes souvent consultées ensemble**
  - Au sein d'une famille de colonnes, les **lignes** peuvent avoir des **colonnes différentes**
  - Chaque **ligne** est **identifiée par une clé unique** et peut être répartie à travers plusieurs familles de colonnes
  - Les colonnes d'une même famille sont **stockées de manière contiguë sur le disque**, **ligne par ligne**
- **Requêtes** via les **clés de ligne**, et **parfois** via les **valeurs** de certaines colonnes
- **Performance optimisée en lecture** pour les **données massives** et **stockage optimisé** pour les **données éparses ou hétérogènes**

Base de données en familles de colonnes

Row Key : CustomerID	Column Family : Identity	
	First name	Last Name
001	Mu Bae	Min
	First name	Last Name
002	Francisco	Vila Nova
	First name	Last Name
003	Lena	Adamcyz

Base de données en familles de colonnes

Row Key : CustomerID	Column Family : Identity
001	**First name** : Mu Bae **Last Name** : Min
002	**First name** : Francisco **Last Name** : Vila Nova
003	**First name** : Lena **Last Name** : Adamcyz

Base de données en familles de colonnes

Row Key : CustomerID	Column Family : Identity
001	**First name** : Mu Bae **Last Name** : Min
002	**First name** : Francisco **Last Name** : Vila Nova
003	**First name** : Lena **Last Name** : Adamcyz

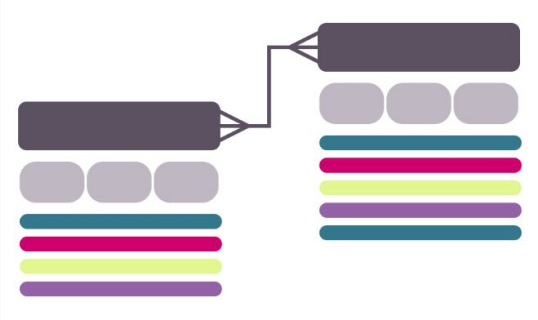
Row Key : CustomerID	Column Family : Contact Info
001	**Phone number** : 555-0100 **Email** : mu-bae.min@example.com
002	**Email** : francisco.vila-nova@example.com **Country** : USA **State** : California **City** : San Francisco **Street** : Kearny St **Num** : 135
003	**Status** : inactive

Base de données en familles de colonnes

Row Key : CustomerID	Column Family : Identity
001	**First name** : Mu Bae **Last Name** : Min
002	**First name** : Francisco **Last Name** : Vila Nova
003	**First name** : Lena **Last Name** : Adamcyz

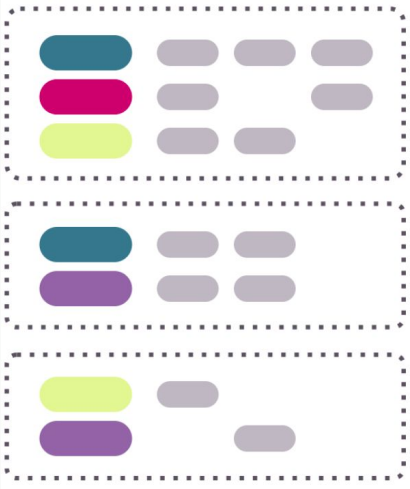
Row Key : CustomerID	Column Family : Contact Info
001	**Phone number** : 555-0100 **Email** : mu-bae.min@example.com
002	**Email** : francisco.vila-nova@example.com **Country** : USA **State** : California **City** : San Francisco **Street** : Kearny St **Num** : 135
003	**Status** : inactive

Row Key : CustomerID	Column Family : Purchase History
002	**Last Purchase** : “2024-02-01” **Oder Count** : 5 **Total Purchase** : 150 **Products** : [P001, P008, P007]



Base de données relationnelles

Base de données clé-valeur



Base de données en colonnes

Base de données orientées documents

Base de données NoSQL : les catégories

Il existe quatre catégories de bases de données NoSQL :

- **Base de données clé-valeur**
  Key-value database
- **Base de données orientées documents**
  Document-oriented database
- **Base de données en familles de colonnes**
  Column-family database, Wide-column database
- **Base de données de graphe**
  Graph database

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée

  Les bases de données de graphe se distingue des autres types de bases de données NoSQL

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :
  - Un **noeud** (ou vertex) est un **objet** concret (ex : personne, produit …)

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :
  - Un **noeud** (ou vertex) est un **objet** concret (ex : personne, produit …)
  - Une **relation** (ou arête) est un **lien** sémantique **entre deux noeuds** (ex : “est ami avec”, “a acheté”, “travaille à”).

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :
  - Un **noeud** (ou vertex) est un **objet** concret (ex : personne, produit …)
  - Une **relation** (ou arête) est un **lien** sémantique **entre deux noeuds** (ex : “est ami avec”, “a acheté”, “travaille à”).
  - Les relations ont une **direction**
  - Les noeuds comme les relations peuvent avoir des **attributs** (ou propriétés) et des **labels**

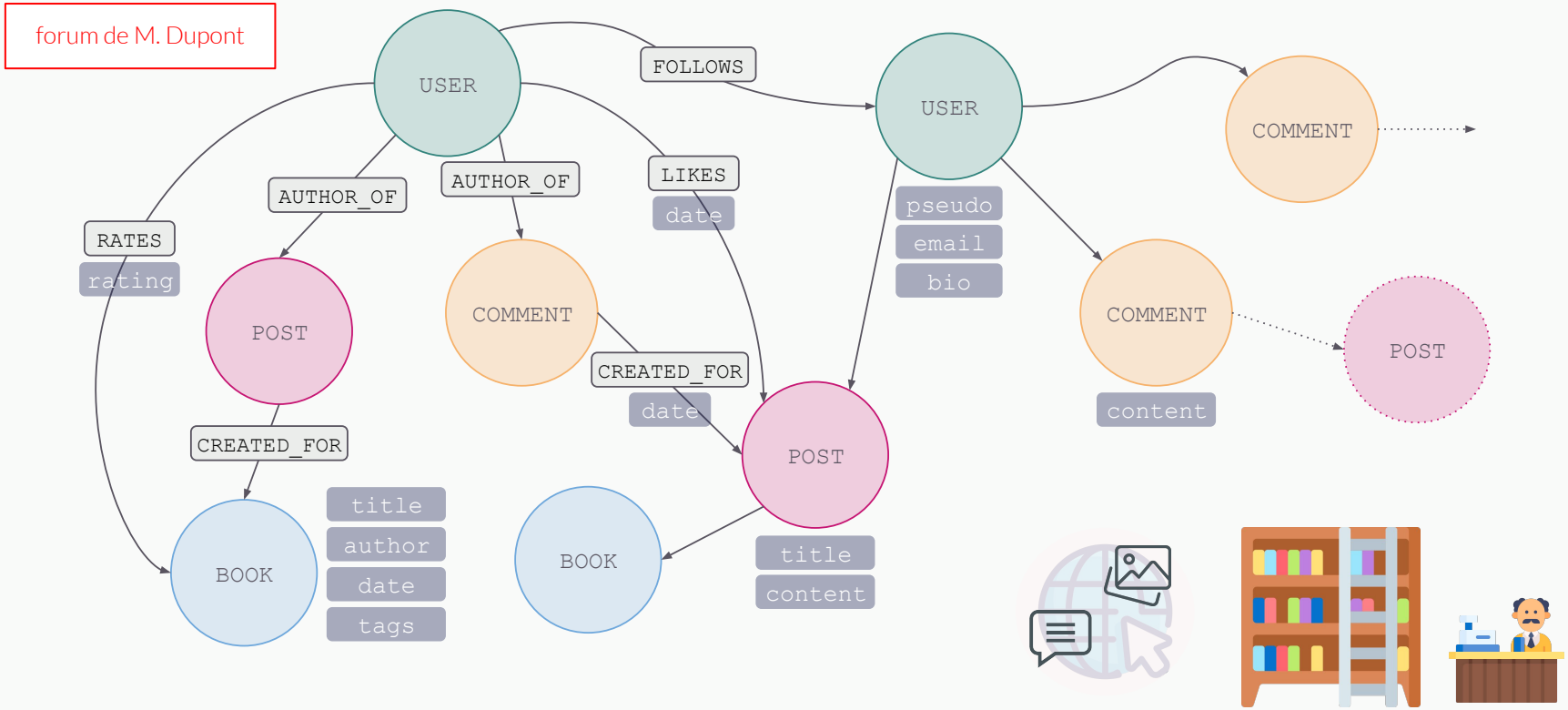
Base de données de graphe

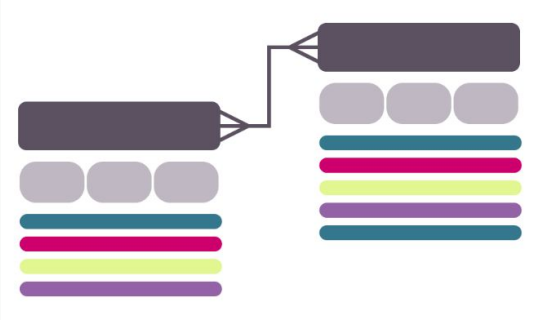
- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :
  - Un **noeud** (ou vertex) est un **objet** concret (ex : personne, produit …)
  - Une **relation** (ou arête) est un **lien** sémantique **entre deux noeuds** (ex : “est ami avec”, “a acheté”, “travaille à”).
  - Les relations ont une **direction**
  - Les noeuds comme les relations peuvent avoir des **attributs** (ou propriétés) et des **labels**
- **Requêtes** via les **attributs** et les **labels** des **noeuds** et des **relations. Navigation** à travers le réseau d’arêtes plutôt que jointures.

Base de données de graphe

- Base de données NoSQL **non orientée agrégats**, scalabilité limitée
- Données organisées en **noeuds** (*nodes*) et en **relations** (*edges*) :
  - Un **noeud** (ou vertex) est un **objet** concret (ex : personne, produit …)
  - Une **relation** (ou arête) est un **lien** sémantique **entre deux noeuds** (ex : “est ami avec”, “a acheté”, “travaille à”).
  - Les relations ont une **direction**
  - Les noeuds comme les relations peuvent avoir des **attributs** (ou propriétés) et des **labels**
- **Requêtes** via les **attributs** et les **labels** des **noeuds** et des **relations. Navigation** à travers le réseau d’arêtes plutôt que jointures.
- **Performance optimisée en lecture** pour les **données fortement interconnectées**, naturellement modélisées sous forme de graphe

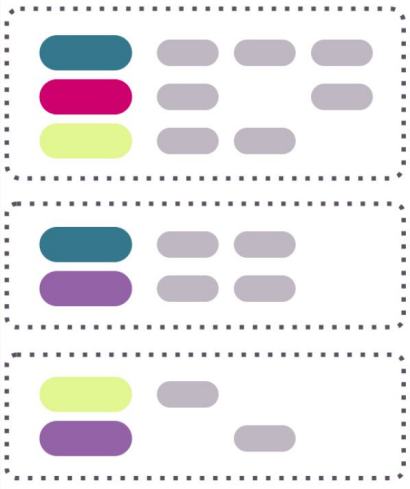
Base de données de graphe



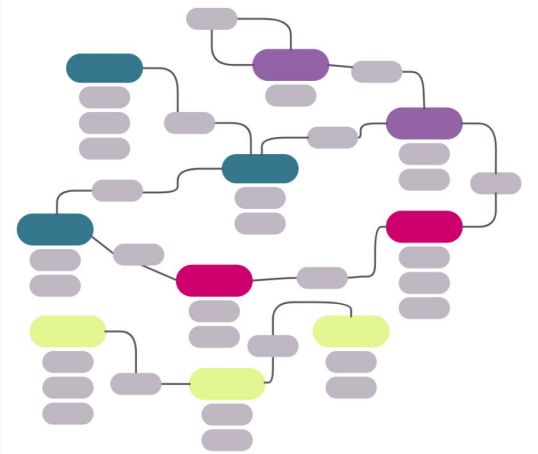


Base de données relationnelles

Base de données clé-valeur



Base de données en colonnes



Base de données de graphe

Base de données orientées documents

MERCI

Choix des agrégats :

- Identifier les informations les plus critiques
  - Quelles seront les **requêtes les plus fréquentes** ? (lecture / écriture)
  - Les **données associées** doivent être **faciles d'accès**
  - Les données associées doivent être groupées de façon à former des **unités autosuffisantes**
- Identifier les **règles qui doivent toujours être respectées** pour garantir l'**intégrité des données** (invariants)
  - Dans la mesure du possible, ces règles doivent pouvoir être **contrôlées au sein de l'agrégat**
- Les agrégats ne doivent **pas** être **trop petits**
  - Coût et complexité de la communication entre agrégats
  - Frein à l'intégrité des données
- Les agrégats ne doivent **pas** être **trop gros** :
  - Amplification des problèmes liés aux accès concurrents à un même agrégat
  - Réduction des performances globales due au transport d'agrégats volumineux (latence, débit, bande passante)
  - Scalabilité limitée par la taille initiale de l'agrégat